

Linux Server Deployment & Configuration

Within VirtualBox

Adriel Sanchez

Summary

This documentation covers the end-to-end process of deploying and hardening a Linux server within a VirtualBox virtualized environment. The project includes OS installation, system updates, tool configuration, user and permission management, firewall hardening, SSH remote access, intrusion prevention with Fail2Ban, GUI setup, Docker container deployment, and real-time network monitoring. Commands and screenshots are provided throughout to support replication and understanding.

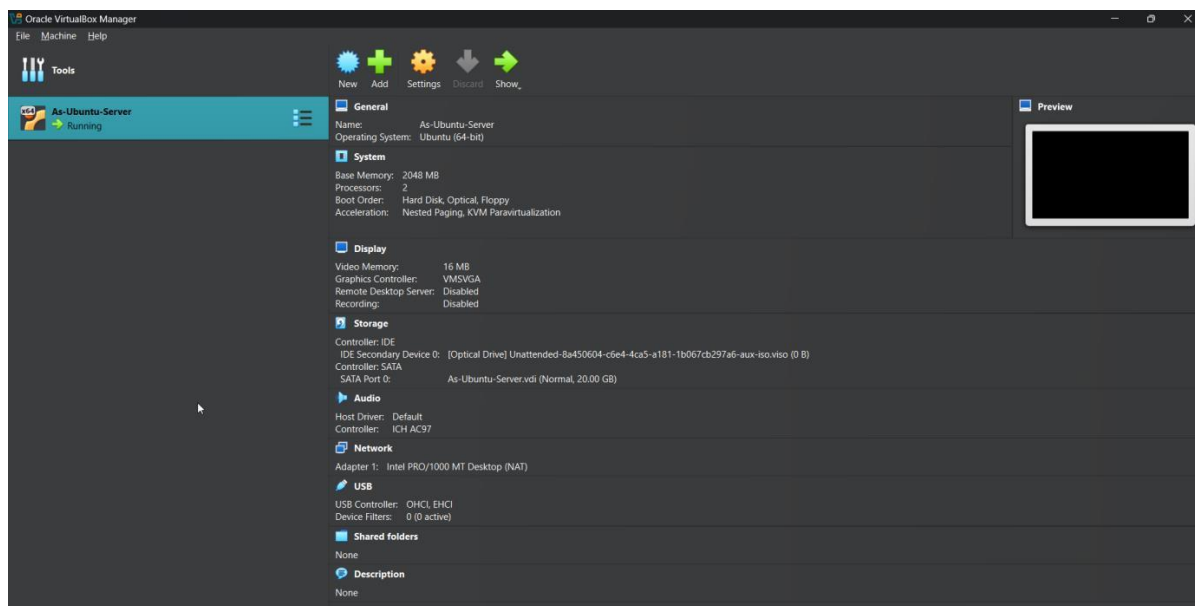
Skills Demonstrated

- Virtualization using VirtualBox
- Linux server installation and baseline configuration
- Package management with APT and repository expansion
- User creation and group-based permission management
- Firewall hardening with UFW
- SSH configuration and remote access via PuTTY
- Intrusion prevention with Fail2Ban
- GUI installation with Cinnamon and LightDM
- Docker installation and container management
- System and container monitoring (Neofetch, htop, Netdata, Portainer)
- Real-time network traffic analysis with Iptraf
- Networking concepts including port forwarding and localhost-only access control

Methodology

Step 1 Environment Setup

Downloaded VirtualBox and the Ubuntu Server 24.04 LTS ISO. Configured the virtual machine with 2GB RAM, 2 virtual CPUs, 20GB storage, and NAT networking to provide a controlled deployment environment.



VirtualBox VM configuration overview

Step 2 System Update & Automatic Security Patching

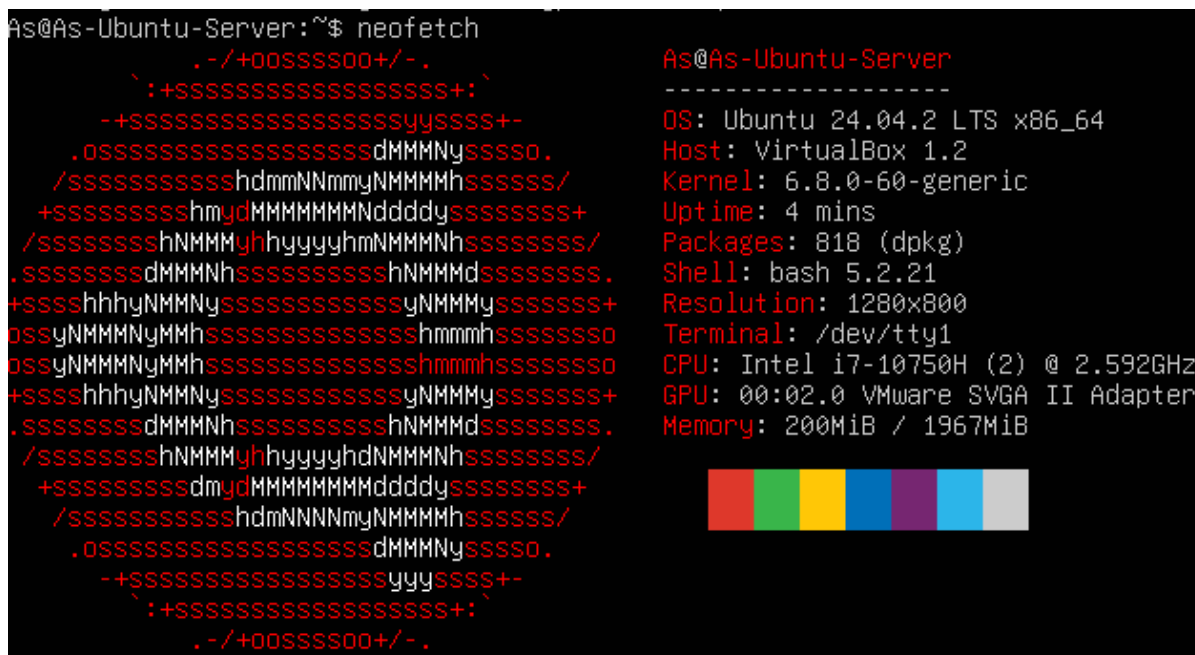
Updated all installed packages to their latest versions and enabled unattended upgrades to ensure the system automatically applies security patches without manual intervention.

```
sudo apt update && sudo apt upgrade -y
sudo apt install unattended-upgrades
sudo dpkg-reconfigure --priority=low unattended-upgrades
```

Step 3 Tool & Repository Installation

Installed essential monitoring, file transfer, and system utilities. Enabled the Multiverse and Universe repositories to expand available software and installed restricted extras for broader codec and driver support.

```
Packages: neofetch, htop, curl, wget, xorg, tldr, dust, restricted-extras
```



Neofetch output confirming Ubuntu 24.04 LTS on VirtualBox

```

0[          0.0%] Tasks: 21, 33 thr, 88 kthr; 1 running
1[          0.0%] Load average: 0.00 0.02 0.00
Mem[|||||] 188M/1.92G Uptime: 00:15:09
Sup[|||||] 0K/0K

Main I/O
PID USER PRI NI VIRT RES SHR S CPU%MEM% TIME+ Command
1 root 20 0 22428 13284 9316 S 0.0 0.7 0:00.78 /sbin/init splash noproct noshell automatic-ubuntu
302 root 19 -1 66828 17020 15996 S 0.0 0.8 0:00.00 /usr/lib/systemd/systemd-journald
356 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.01 /sbin/multipathd -d -s
366 root 20 0 29216 7808 4992 S 0.0 0.4 0:00.10 /usr/lib/systemd/systemd-udev
368 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.00 /sbin/multipathd -d -s
369 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.00 /sbin/multipathd -d -s
370 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.00 /sbin/multipathd -d -s
371 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.00 /sbin/multipathd -d -s
372 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.04 /sbin/multipathd -d -s
373 root 20 0 282M 27136 8704 S 0.0 1.3 0:00.00 /sbin/multipathd -d -s
419 systemd-re 20 0 21580 12800 10624 S 0.0 0.6 0:00.06 /usr/lib/systemd/systemd-resolved
451 systemd-ne 20 0 18992 9472 8320 S 0.0 0.5 0:00.02 /usr/lib/systemd/systemd-networkd
581 messagebus 20 0 9772 5376 4608 S 0.0 0.3 0:00.11 @dbus-daemon --system --address=systemd: --nofork --nopidfile --systemd-activation --syslog-o
588 polkitd 20 0 374M 9744 7424 S 0.0 0.5 0:00.03 /usr/lib/polkit-1/polkitd --no-debug
599 root 20 0 18124 8704 7680 S 0.0 0.4 0:00.05 /usr/lib/systemd/systemd-logind
602 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.03 /usr/libexec/udisks2/udisksd
621 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.00 /usr/libexec/udisks2/udisksd
622 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.00 /usr/libexec/udisks2/udisksd
625 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.00 /usr/libexec/udisks2/udisksd
632 root 20 0 107M 22912 13568 S 0.0 1.1 0:00.06 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
644 syslog 20 0 217M 6144 4608 S 0.0 0.3 0:00.01 /usr/sbin/rsyslogd -n -INONE
646 polkitd 20 0 374M 9744 7424 S 0.0 0.5 0:00.00 /usr/lib/polkit-1/polkitd --no-debug
647 polkitd 20 0 374M 9744 7424 S 0.0 0.5 0:00.00 /usr/lib/polkit-1/polkitd --no-debug
648 polkitd 20 0 374M 9744 7424 S 0.0 0.5 0:00.00 /usr/lib/polkit-1/polkitd --no-debug
661 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.00 /usr/libexec/udisks2/udisksd
667 root 20 0 382M 12928 10880 S 0.0 0.6 0:00.05 /usr/sbin/ModemManager
686 root 20 0 457M 13312 11264 S 0.0 0.7 0:00.00 /usr/libexec/udisks2/udisksd
690 root 20 0 382M 12928 10880 S 0.0 0.6 0:00.00 /usr/sbin/ModemManager
692 syslog 20 0 217M 6144 4608 S 0.0 0.3 0:00.00 /usr/sbin/rsyslogd -n -INONE
693 syslog 20 0 217M 6144 4608 S 0.0 0.3 0:00.00 /usr/sbin/rsyslogd -n -INONE
694 syslog 20 0 217M 6144 4608 S 0.0 0.3 0:00.00 /usr/sbin/rsyslogd -n -INONE
696 root 20 0 382M 12928 10880 S 0.0 0.6 0:00.00 /usr/sbin/ModemManager
698 root 20 0 382M 12928 10880 S 0.0 0.6 0:00.00 /usr/sbin/ModemManager
720 root 20 0 107M 22912 13568 S 0.0 1.1 0:00.00 /usr/bin/python3 /usr/share/unattended-upgrades/unattended-upgrade-shutdown --wait-for-signal
969 root 20 0 280M 2560 2432 S 0.0 0.1 0:00.10 /usr/bin/VBoxDRMClient
971 root 20 0 286M 3596 3200 S 0.0 0.2 0:00.00 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.sh
975 root 20 0 280M 2560 2432 S 0.0 0.1 0:00.00 /usr/bin/VBoxDRMClient
976 root 16 -4 280M 2560 2432 S 0.0 0.1 0:00.06 /usr/bin/VBoxDRMClient
977 root 20 0 286M 3596 3200 S 0.0 0.2 0:00.00 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.sh
978 root 20 0 286M 3596 3200 S 0.0 0.2 0:00.00 /usr/sbin/VBoxService --pidfile /var/run/vboxadd-service.sh
979 root 20 0 280M 2560 2432 S 0.0 0.1 0:00.02 /usr/bin/VBoxDRMClient

```

htop showing active processes and system resource usage

Step 4 User & Permission Management

Created two additional user accounts and assigned them to appropriate groups to simulate real-world access control. This enforces the principle of least privilege by separating standard users from administrative accounts.

```

As@As-Ubuntu-Server:~$ groups
As sudo vboxsf
As@As-Ubuntu-Server:~$ groups user1
user1 : user1 users
As@As-Ubuntu-Server:~$ groups user2
user2 : user2 guest

```

Group membership output for primary user, user1, and user2

Step 5 VirtualBox Guest Additions

Installed VirtualBox Guest Additions to enable clipboard sharing, shared folder access, and improved display resolution between the host and virtual machine.

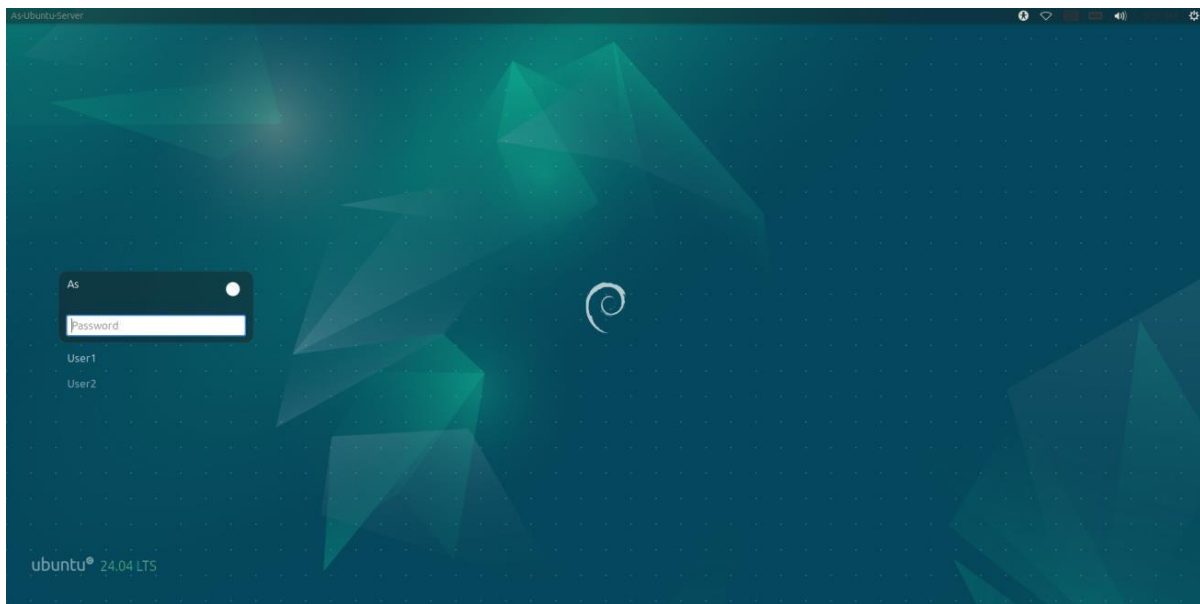
```

sudo apt install build-essential dkms linux-headers-$(uname -r)
sudo mkdir -p /mnt/cdrom
sudo mount /dev/cdrom /mnt/cdrom
sudo sh /mnt/cdrom/VBoxLinuxAdditions.run

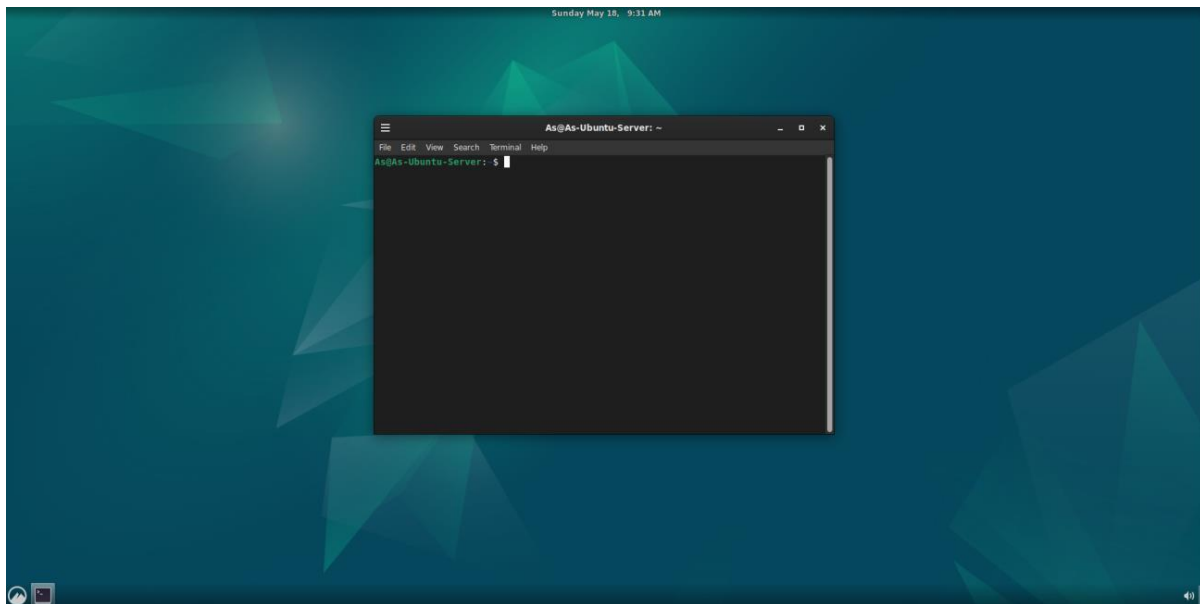
```

Step 6 GUI Installation (Cinnamon + LightDM)

Installed the Cinnamon desktop environment and LightDM display manager to provide a graphical interface for easier server management without relying solely on the command line.



LightDM login screen with user accounts visible



Cinnamon desktop environment running on the Ubuntu Server VM

Step 7 Firewall Configuration (UFW)

Configured UFW to deny all incoming connections by default, allow all outgoing traffic, and permit SSH only from the local network range. Logging was enabled to maintain an audit trail of network activity.

```
As@As-Ubuntu-Server: ~
File Edit View Search Terminal Help
As@As-Ubuntu-Server:~$ sudo ufw status verbose

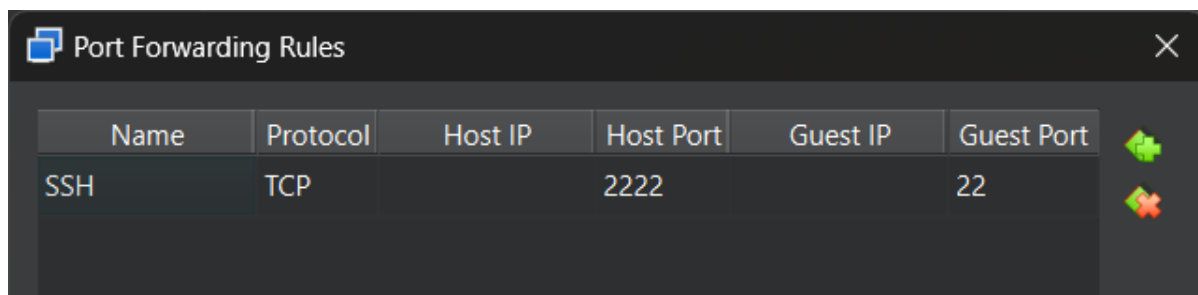
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22 ALLOW IN 10.0.0.0/8
```

UFW verbose status confirming deny-incoming policy and SSH allow rule

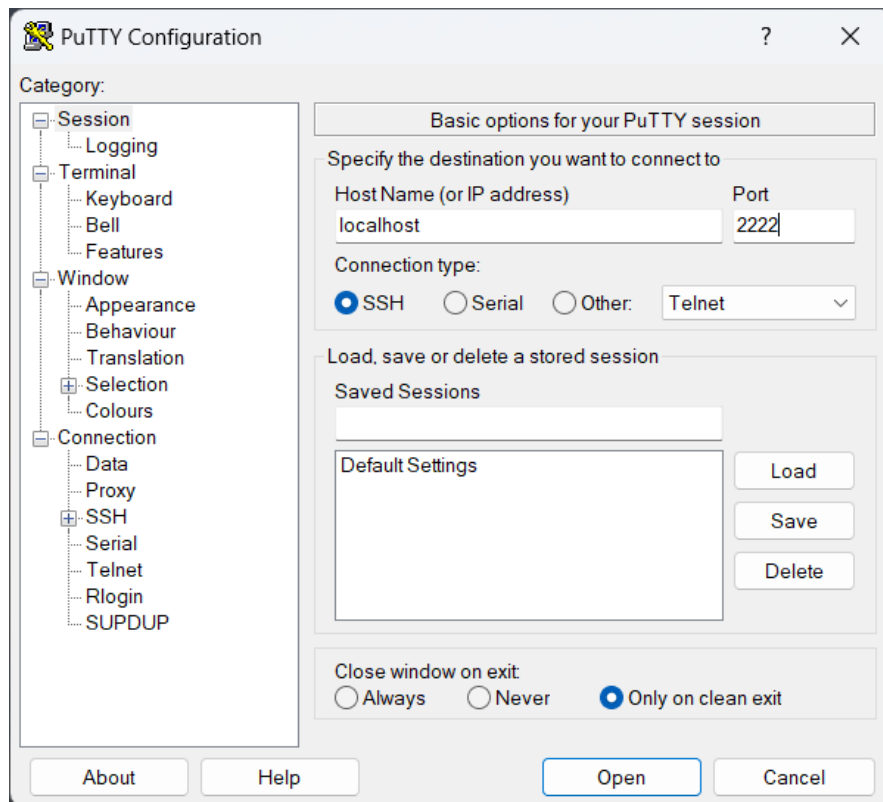
Step 8 SSH Remote Access via PuTTY

Installed OpenSSH Server and configured VirtualBox NAT port forwarding to map host port 2222 to guest port 22. Used PuTTY on the host machine to establish an encrypted SSH session into the VM for remote management.



Name	Protocol	Host IP	Host Port	Guest IP	Guest Port
SSH	TCP		2222		22

VirtualBox NAT port forwarding rule: host 2222 → guest 22



PuTTY configured to connect to localhost:2222 via SSH

Step 9 Intrusion Prevention with Fail2Ban

Installed and configured Fail2Ban to monitor SSH login attempts and automatically ban IP addresses that exceed the failure threshold. The jail was configured to block IPs after 5 failed attempts within a 5-minute window, with a 5-minute ban duration.

```
sudo apt install fail2ban -y
sudo cp /etc/fail2ban/jail.conf /etc/fail2ban/jail.local
sudo nano /etc/fail2ban/jail.local
```

```
As@As-Ubuntu-Server: ~
login as: As
As@localhost's password:
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 6.8.0-60-generic x86_64)

* Documentation:  https://help.ubuntu.com
* Management:    https://landscape.canonical.com
* Support:       https://ubuntu.com/pro

System information as of Fri May 16 01:43:41 AM UTC 2025

System load:          0.0
Usage of /:           18.6% of 19.51GB
Memory usage:         12%
Swap usage:           0%
Processes:            116
Users logged in:      1
IPv4 address for enp0s3: 10.0.2.15
IPv6 address for enp0s3: fd17:625c:f037:2:a00:27ff:fe24:afdc

* Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
  just raised the bar for easy, resilient and secure K8s cluster deployment.

  https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

11 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

Last login: Fri May 16 01:42:38 2025 from 10.0.2.2
As@As-Ubuntu-Server:~$
```

jail.local SSH configuration: maxretry=5, findtime=300, bantime=300

```
[sshd]
enabled=true
port = ssh
logpath = %(sshd_log)s
maxretry = 5
findtime = 300
bantime = 300
backend = systemd
#Enables SSH monitor (port 22) & ban Failed login attempts afterfor 5 min after 5 tries
```

Fail2Ban SSHD jail configuration snippet

Intentionally entered incorrect credentials via PuTTY to trigger the Fail2Ban rule. Confirmed the source IP was banned by querying the SSHD jail status on the server.

```
ASPC2 - PuTTY
login as:
@localhost's password:
Access denied
@localhost's password:
Access denied
@localhost's password:
Access denied
@localhost's password:
Access denied
@localhost's password:
```

PuTTY showing repeated access denied after failed SSH attempts

```

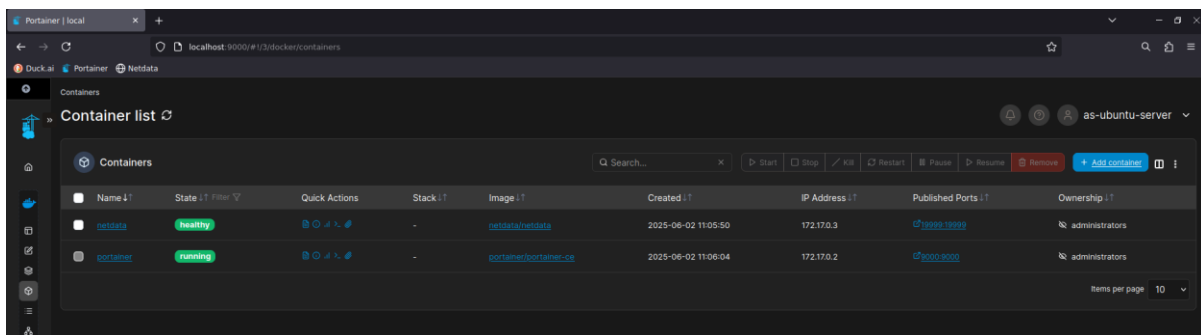
As@As-Ubuntu-Server:~$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
|   |- Currently failed: 0
|   |- Total failed:     5
|   \- Journal matches:  _SYSTEMD_UNIT=sshd.service + _COMM=sshd
|- Actions
|   |- Currently banned: 1
|   |- Total banned:     1
|   \- Banned IP list:   10.0.2.2
As@As-Ubuntu-Server:~$

```

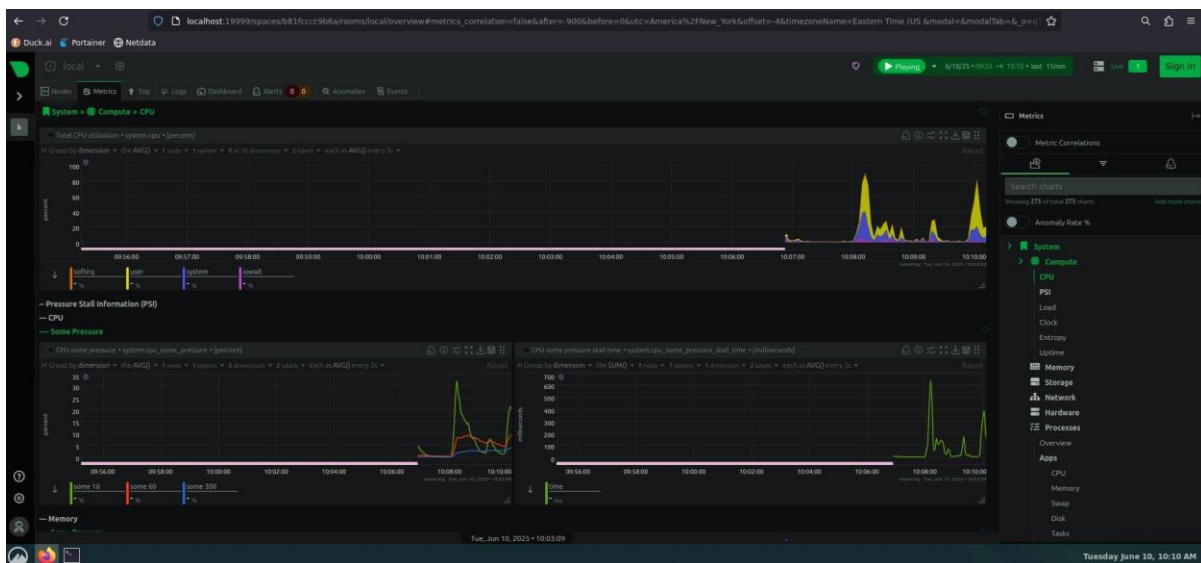
Fail2Ban status confirming IP 10.0.2.2 banned after 5 failures

Step 10 Docker & Container Deployment

Installed Docker and deployed two containers: Portainer for graphical container management and Netdata for real-time system and performance monitoring. Both containers were configured to run persistently and restart automatically on system reboot.



Portainer container management UI showing Netdata and Portainer running



Netdata real-time CPU and system performance dashboard

Step 11 Container Access Restriction

Updated UFW rules to restrict access to the Portainer (port 9000) and Netdata (port 19999) container interfaces to localhost only, preventing external network exposure of the management dashboards.


```
As@As-Ubuntu-Server: ~
File Edit View Search Terminal Help
As@As-Ubuntu-Server:~$ sudo ufw status verbose

Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), deny (routed)
New profiles: skip

To Action From
--
22 ALLOW IN 10.0.0.0/8

19999 ALLOW IN 127.0.0.1

9000 ALLOW IN 127.0.0.1
```

UFW rules updated to allow container ports from 127.0.0.1 only

Step 12 Real-Time Network Monitoring with Iptraf

Installed Iptraf-ng to enable real-time packet-level network traffic monitoring, providing visibility into incoming and outgoing traffic by protocol, interface, and connection.

```
As@As-Ubuntu-Server: ~
File Edit View Search Terminal Help
iptraf-ng 1.2.1
Statistics for enp0s3

Total      Total      Incoming  Incoming  Outgoing  Outgoing
Packets    Bytes      Packets   Bytes     Packets   Bytes
Total:      23         2353      9         994       14        1359
IPv4:       20         2057      8         890       12        1167
IPv6:       3          288       1         96        2         192
TCP:        8          813       2         84        6         729
UDP:       12         1244      6         806       6         438
ICMP:       1          96        1         96        0         0
Other IP:   2          192       0         0         2         192
Non-IP:     0          0         0         0         0         0
Broadcast:  0          0         0         0         0         0

Total rates:      2.11 kbps      Broadcast rates:  0.00 kbps
                  1 pps                        0 pps

Incoming rates:   0.72 kbps
                  0 pps

Outgoing rates:   1.38 kbps
                  0 pps

IP checksum errors: 0

Time: 0:02 Drops: 0
X-exit
```

Iptraf-ng displaying live packet statistics for enp0s3

After Action Summary

This project successfully demonstrated a complete Linux server deployment lifecycle within a virtualized environment. Beginning with OS installation and system hardening, the configuration progressed through user management, GUI setup, firewall policy enforcement, encrypted remote access, and automated intrusion prevention. Docker containers were deployed for both management and monitoring purposes, with access controls applied to prevent external exposure. The project established a repeatable, security-conscious baseline that reflects real-world system administration and security operations practices.